

LAPORAN TUGAS KELOMPOK 5

“Hardening Keamanan Transmisi Data REST API Berbasis Docker Container Menggunakan TLS 1.3 Berpola Cipher Suite AES-128-GCM Terhadap Serangan Adversary-in-the-Middle”

Mata Kuliah Network Programming & Administration

Kelas IFN41



Kelompok 5:

**Marsani (230401010282), Muhammad Saifulloh (220401010207),
Kristian Hananiel Hura (220401010289), Sukandar (240401020175)**

Dosen Pengajar:

Abdul Azzam Ajhari, S.Kom., M.Kom

PROGRAM STUDI PJJ INFORMATIKA

UNIVERSITAS SIBER ASIA

2026

Project-2 Kelompok 5

Hardening Transmisi Data REST API Terhadap Serangan Adversary-in-the-Middle

Mata Kuliah: Network Programming & Administration, Kelas: IFN41

Kelompok 5:

- **Marsani (230401010282)**
- **Muhammad Saifulloh (220401010207)**
- **Kristian Hananiel Hura (220401010289)**
- **Sukandar (240401020175)**

Dosen Pengajar: Abdul Azzam Ajhari, S.Kom., M.Kom

**PJJ INFORMATIKA
UNIVERSITAS SIBER ASIA
2026**

Daftar Isi

Daftar Isi.....	1
1. Pendahuluan.....	3
1.1 Latar Belakang.....	3
1.2 Rumusan Masalah.....	4
1.3 Tujuan Penelitian.....	4
1.4 Batasan Masalah.....	5
1.5 Manfaat Penelitian.....	5
2. Tinjauan Pustaka.....	6
2.1 Penelitian Terkait.....	6
2.1.1 Jurnal Referensi 1.....	6
2.1.2 Jurnal Referensi 2.....	6
2.1.3 Posisi Penelitian Ini (Novelty).....	7
2.2 Landasan Teori.....	8
2.2.1 REST API dan Kerentanan Layer Transport.....	8
2.2.2 Transport Layer Security (TLS) 1.3.....	8
2.2.3 AES-128-GCM (Galois/Counter Mode).....	9
2.2.4 Nginx sebagai TLS Termination Reverse Proxy.....	9
2.2.5 Docker Container Networking.....	9
2.2.6 Framework MITRE ATT&CK.....	10
2.2.7 OWASP Top 10:2025.....	10
3. Metodologi Penelitian.....	11
3.1 Jenis dan Pendekatan Penelitian.....	11
3.2 Alat dan Bahan.....	11
3.2.1 Perangkat Keras.....	11
3.2.2 Perangkat Lunak.....	11

3.3 Arsitektur Sistem.....	12
3.4 Implementasi.....	14
3.4.1 Langkah 1: Pembuatan Self-Signed TLS Certificate.....	14
3.4.2 Langkah 2: Konfigurasi Backend API.....	14
3.4.3 Langkah 3: Konfigurasi Nginx Reverse Proxy.....	16
3.4.4 Langkah 4: Orkestrasi dengan Docker Compose.....	17
3.4.5 Langkah 5: Build dan Jalankan Environment.....	18
3.4.6 Langkah 6: Verifikasi Koneksi TLS.....	18
3.5 Prosedur Pengujian.....	19
Skenario A: Baseline - Transmisi HTTP Tanpa Enkripsi.....	19
Skenario B: Pasca-Hardening - Transmisi HTTPS dengan TLS 1.3.....	20
3.6 Metrik Evaluasi.....	20
4. Hasil dan Pembahasan.....	21
4.1 Hasil Implementasi Arsitektur.....	21
4.2 Hasil Pengujian Skenario A: Baseline HTTP.....	22
4.3 Hasil Pengujian Skenario B: Pasca-Hardening TLS 1.3.....	22
4.4 Analisis Komparatif Before-After.....	23
4.5 Pemetaan Terhadap MITRE ATT&CK.....	24
4.6 Pemetaan Terhadap OWASP Top 10:2025.....	25
4.7 Perbandingan dengan Penelitian Sebelumnya.....	26
5. Kesimpulan dan Future Work.....	27
5.1 Kesimpulan.....	27
5.2 Future Work.....	28
Daftar Pustaka.....	29

1. Pendahuluan

1.1 Latar Belakang

Sebagaimana yang telah umum diketahui bahwa arsitektur layanan berbasis REST API telah menjadi fondasi utama dalam pengembangan aplikasi modern, termasuk pada arsitektur *microservices*. Komunikasi antar-layanan dalam arsitektur tersebut bergantung pada protokol HTTP sebagai medium transmisi data. Namun, HTTP secara inheren tidak menyediakan mekanisme enkripsi pada layer transport, sehingga seluruh data yang ditransmisikan, termasuk kredensial autentikasi, token otorisasi, dan payload sensitif, dapat diintersepsi oleh pihak ketiga melalui teknik *network sniffing*.

Serangan *Adversary-in-the-Middle* (AitM), atau yang sebelumnya dikenal sebagai *Man-in-the-Middle* (MITM), merupakan salah satu vektor ancaman utama terhadap transmisi data yang tidak terenkripsi. Pada serangan ini, aktor ancaman (*threat actor*) memposisikan dirinya di tengah-tengah antara dua entitas komunikasi untuk menyadap, memodifikasi, atau menginjeksi data tanpa sepengetahuan kedua pihak. Framework MITRE ATT&CK mengkategorikan teknik ini sebagai *Network Sniffing* (T1557.001) di bawah taktik *Credential Access* dan *Collection*. Secara paralel, OWASP Top 10:2025 juga mengklasifikasikan kegagalan dalam mengamankan transmisi data sebagai bagian dari kategori A02:2025-*Cryptographic Failures*.

Penerapan HTTPS melalui protokol TLS merupakan langkah mitigasi standar terhadap ancaman tersebut. Namun, tidak semua implementasi TLS dapat memberikan tingkat keamanan yang setara. Versi TLS yang lebih lama (TLS 1.0, 1.1, dan sebagian konfigurasi TLS 1.2) masih berpeluang rentan terhadap serangan *downgrade*, *padding oracle*, dan eksploitasi *cipher suite* yang lemah seperti RC4, DES, atau CBC-mode tanpa mekanisme *Authenticated Encryption*. Adapun TLS 1.3, yang diratifikasi melalui RFC 8446 pada Agustus 2018, mengeliminasi seluruh *cipher suite* dan mekanisme *key exchange* yang dianggap tidak aman, serta menyederhanakan proses *handshake* dari dua *round-trip* menjadi satu.

Dalam konteks *containerized deployment*, penggunaan Docker sebagai platform orkestrasi layanan menambahkan dimensi baru pada perimeter keamanan. Komunikasi antar-container dalam jaringan Docker internal secara default tidak terenkripsi, sehingga aktor ancaman yang berhasil mengakses jaringan internal Docker dapat melakukan intersepsi data secara langsung. Oleh karena itu, diperlukan arsitektur keamanan yang menempatkan *reverse proxy* sebagai titik terminasi TLS (*TLS termination point*) untuk memastikan seluruh komunikasi eksternal melewati kanal terenkripsi.

Penelitian ini mengusulkan arsitektur *hardening* keamanan transmisi data REST API menggunakan Nginx *reverse proxy* yang dikonfigurasi secara restriktif untuk hanya menerima koneksi TLS 1.3 dengan *cipher suite* AES-128-GCM (TLS_AES_128_GCM_SHA256) dalam ekosistem Docker container. Pendekatan ini bertujuan untuk mengeliminasi permukaan serangan pada layer transport dan memvalidasi efektivitasnya melalui pengujian berbasis framework MITRE ATT&CK dan OWASP Top 10:2025.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana arsitektur keamanan Defense in Depth berlapis TLS 1.3 (Data in Transit) dan enkripsi tingkat aplikasi AES-128-GCM (Data at Rest) dapat diimplementasikan pada aplikasi CRUD REST API dalam ekosistem Docker container?
2. Bagaimana efektivitas konfigurasi tersebut dalam mencegah serangan Adversary-in-the-Middle berupa network sniffing dan melindungi kerahasiaan data dari insiden pasca-kebocoran (post-breach) pada sisi database?
3. Bagaimana pemetaan hasil pengujian terhadap framework MITRE ATT&CK (T1557.001) dan OWASP Top 10:2025 (A02:2025) untuk memvalidasi mitigasi yang diterapkan?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini meliputi:

1. Merancang dan mengimplementasikan arsitektur keamanan berlapis (Defense in Depth) pada aplikasi CRUD REST API berbasis Docker container yang mencakup Frontend Web, Backend API (Python), MySQL Database, dan Nginx reverse proxy (TLS 1.3).
2. Menguji efektivitas pengamanan lapis ganda (TLS 1.3 untuk Data in Transit dan AES-128-GCM untuk Data at Rest) dalam mencegah intersepsi data melalui teknik network sniffing menggunakan Wireshark maupun eksfiltrasi langsung pada tingkat database.
3. Memetakan hasil pengujian ke dalam framework MITRE ATT&CK dan OWASP Top 10:2025 sebagai bentuk validasi terhadap standar keamanan industri.

1.4 Batasan Masalah

Penelitian ini membatasi ruang lingkupnya pada aspek-aspek berikut:

1. Pengujian dan simulasi arsitektur microservices dilakukan secara terisolasi di dalam lingkungan jaringan virtual Docker (*Docker Custom Bridge Network*) berskala *single-node*, bukan didistribusikan pada kluster infrastruktur *cloud* (seperti Kubernetes) atau lingkungan produksi nyata.
2. Sertifikat TLS yang digunakan bersifat self-signed, yang memadai untuk keperluan pengujian namun tidak merepresentasikan skenario Certificate Authority (CA) publik.
3. Fokus pengujian ancaman terbatas pada serangan network sniffing (T1557.001) dan tidak mencakup vektor serangan aktif seperti SSL stripping, SQL Injection, atau side-channel attack.
4. Backend API menggunakan framework Python (Flask) dan MySQL Database yang difokuskan pada fungsionalitas CRUD data karyawan untuk mendemonstrasikan enkripsi tingkat aplikasi.
5. Pengujian Data at Rest difokuskan pada simulasi mitigasi pasca-kompromi dengan memverifikasi ciphertext secara langsung di level database.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan kontribusi sebagai berikut:

1. Manfaat Teoretis: Memperkaya literatur mengenai implementasi *hardening* TLS 1.3 pada arsitektur *microservices* berbasis Docker, khususnya dalam konteks pemilihan *cipher suite* yang restriktif.
2. Manfaat Praktis: Menyediakan panduan teknis (*technical guideline*) bagi pengembang dan administrator jaringan dalam mengkonfigurasi Nginx sebagai *TLS termination proxy* dengan standar keamanan yang sesuai dengan rekomendasi MITRE dan OWASP.

2. Tinjauan Pustaka

2.1 Penelitian Terkait

2.1.1 Jurnal Referensi 1

Penelitian yang dilakukan oleh Permana & Ramadhan dalam jurnal berjudul "*Analisis Keamanan Jaringan pada Layanan WiFi dengan Menggunakan Wireshark*" (2021) mengkaji kerentanan protokol HTTP pada jaringan publik. Penelitian tersebut melakukan analisis terhadap paket data yang ditransmisikan melalui HTTP menggunakan Wireshark sebagai *packet analyzer*. Hasil pengujian menunjukkan bahwa kredensial pengguna berupa *username* dan *password* yang dikirimkan melalui HTTP dapat diintersepsi secara utuh dalam format *cleartext* menggunakan fitur *Follow TCP Stream* pada Wireshark. Sebagai rekomendasi, penelitian tersebut menyarankan pentingnya enkripsi keamanan jaringan.

Keterbatasan yang teridentifikasi: Penelitian tersebut berhenti pada tahap identifikasi kerentanan dan rekomendasi generik tanpa melakukan implementasi konkret terhadap solusi yang diusulkan. Tidak terdapat pembahasan mengenai versi TLS yang seharusnya digunakan, pemilihan *cipher suite*, maupun arsitektur *deployment* yang aman.

2.1.2 Jurnal Referensi 2

Penelitian yang dilakukan oleh Darmawan, Ningsih, & Wahidin dalam jurnal berjudul "*Analisis Keamanan Jaringan terhadap Sniffing Menggunakan Wireshark*" (2024) membahas eksploitasi data *cleartext* pada jaringan melalui serangan *sniffing*. Penelitian tersebut mendemonstrasikan

bagaimana paket data yang ditransmisikan tanpa enkripsi dapat disadap oleh aktor ancaman yang memonitor segmen jaringan. Fokus utama penelitian adalah pada pembuktian kerentanan (*proof of vulnerability*) terhadap protokol yang tidak menerapkan enkripsi berlapis.

Keterbatasan yang teridentifikasi: Serupa dengan jurnal referensi pertama, penelitian ini bersifat deskriptif-eksploratif dan tidak menawarkan solusi teknis yang terukur. Pembahasan terbatas pada demonstrasi serangan tanpa menyertakan mekanisme mitigasi spesifik, pengujian pasca-mitigasi, atau pemetaan terhadap framework keamanan standar.

2.1.3 Posisi Penelitian Ini (Novelty)

Penelitian ini mengambil posisi sebagai kelanjutan dan perbaikan dari kedua jurnal referensi di atas. Jika kedua penelitian sebelumnya berfokus pada identifikasi dan demonstrasi kerentanan (*offensive perspective*), penelitian ini bergerak ke arah mitigasi aktif dan validasi (*defensive perspective*). Secara spesifik, kontribusi yang membedakan penelitian ini dari penelitian sebelumnya adalah:

Aspek	Permana & Ramadhan (2021)	Darmawan et al. (2024)	Penelitian Ini
Fokus Utama	Identifikasi kerentanan HTTP	Demonstrasi sniffing jaringan	Hardening & validasi mitigasi
Solusi yang Ditawarkan	Rekomendasi enkripsi (generik)	Tidak ada solusi eksplisit	TLS 1.3 + AES-128-GCM (spesifik)
Implementasi	Tidak ada	Tidak ada	Docker + Nginx reverse proxy
Pemetaan Framework	Tidak ada	Tidak ada	MITRE ATT&CK + OWASP Top 10:2025

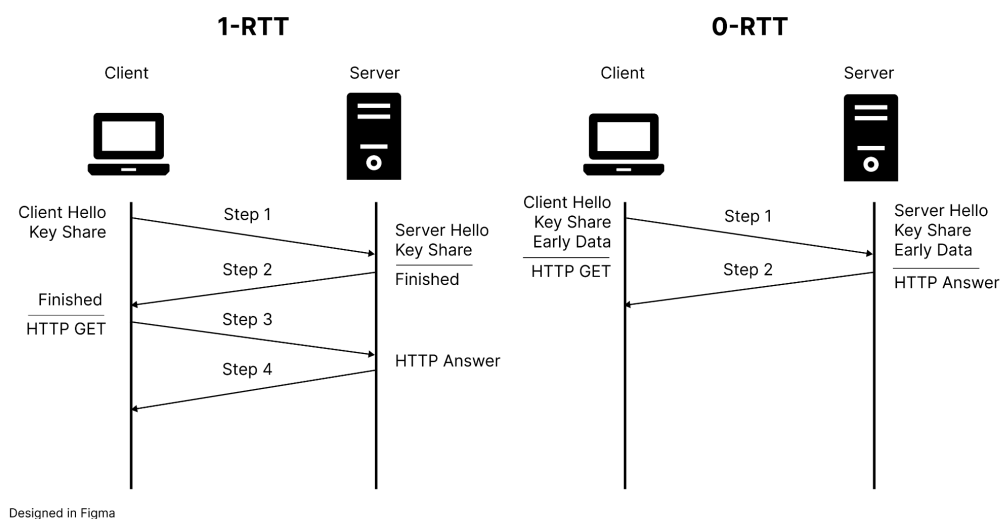
Pengujian Pasca-Mitigasi	Tidak ada	Tidak ada	Wireshark (before-after comparison)
Arsitektur Deployment	Tidak dibahas	Tidak dibahas	3-tier container (client, API, proxy)

2.2 Landasan Teori

2.2.1 REST API dan Kerentanan Layer Transport

Representational State Transfer (REST) merupakan gaya arsitektur perangkat lunak yang mendefinisikan sekumpulan *constraints* untuk perancangan *web service*. REST API berkomunikasi melalui protokol HTTP dengan menggunakan metode standar seperti GET, POST, PUT, dan DELETE. Secara default, komunikasi REST API melalui HTTP berjalan pada port 80 tanpa enkripsi, sehingga seluruh *request* dan *response*, termasuk *header* autentikasi, *cookies*, dan *body* payload, dapat dibaca oleh siapa pun yang memiliki akses ke segmen jaringan yang dilalui paket data tersebut.

2.2.2 Transport Layer Security (TLS) 1.3



Gambar TLS Handshake 1-RTT dan 0-RTT

TLS 1.3, sebagaimana didefinisikan dalam RFC 8446, merupakan revisi mayor dari protokol TLS yang menghapus dukungan terhadap algoritma dan mekanisme yang telah terbukti tidak aman. Perbedaan fundamental antara TLS 1.3 dan versi pendahulunya meliputi:

1. Penghapusan cipher suite lemah: TLS 1.3 hanya mendukung *Authenticated Encryption with Associated Data* (AEAD), yaitu AES-128-GCM, AES-256-GCM, dan ChaCha20-Poly1305. Algoritma seperti RC4, 3DES, dan CBC-mode dihapus sepenuhnya.
2. Penyederhanaan handshake: Proses *handshake* dikurangi dari 2-RTT (TLS 1.2) menjadi 1-RTT, dengan dukungan 0-RTT untuk koneksi berulang.
3. Forward Secrecy wajib: Seluruh *key exchange* menggunakan Ephemeral Diffie-Hellman (DHE atau ECDHE), sehingga kompromi terhadap *private key* server tidak membahayakan sesi komunikasi sebelumnya.
4. Penghapusan fitur rentan: Kompresi TLS, renegotiasi, dan *static RSA key exchange* dihapus untuk menutup vektor serangan seperti CRIME, BREACH, dan *Bleichenbacher attack*.

2.2.3 AES-128-GCM (Galois/Counter Mode)

AES-128-GCM merupakan *cipher suite* yang menggabungkan enkripsi simetris AES dengan panjang kunci 128-bit dan mode operasi GCM yang menyediakan *authenticated encryption*. GCM beroperasi dengan mengombinasikan *Counter Mode* (CTR) untuk enkripsi dengan fungsi hash berbasis *Galois field* untuk autentikasi, menghasilkan *authentication tag* yang memverifikasi integritas dan autentisitas data secara simultan. Dalam konteks TLS 1.3, *cipher suite* ini diidentifikasi sebagai TLS_AES_128_GCM_SHA256 (0x13,0x01) dan merupakan *cipher suite* wajib (*mandatory-to-implement*) sesuai RFC 8446 Section 9.1.

2.2.4 Nginx sebagai TLS Termination Reverse Proxy

Nginx, dalam konfigurasi *reverse proxy*, berfungsi sebagai titik masuk tunggal (*single entry point*) yang menerima koneksi HTTPS dari klien, melakukan terminasi TLS, dan meneruskan *request* ke *backend server* melalui koneksi HTTP internal. Pendekatan ini memiliki beberapa keunggulan arsitektural:

1. Sentralisasi manajemen sertifikat: Sertifikat TLS hanya perlu dikelola pada satu titik, bukan pada setiap *backend service*.
2. Isolasi perimeter keamanan: *Backend service* tidak perlu mengimplementasikan logika TLS, sehingga mengurangi kompleksitas dan potensi miskonfigurasi.
3. Konfigurasi granular: Nginx menyediakan direktif `ssl_protocols` dan `ssl_ciphers` yang memungkinkan pembatasan ketat terhadap versi TLS dan *cipher suite* yang diterima.

2.2.5 Docker Container Networking

Docker menyediakan beberapa *driver* jaringan untuk komunikasi antar-container. Dalam konfigurasi default menggunakan *bridge network*, container yang terhubung pada jaringan yang sama dapat berkomunikasi satu sama lain melalui nama container sebagai *hostname*. Komunikasi internal ini secara default tidak terenkripsi, yang menjadi pertimbangan penting dalam desain arsitektur keamanan. Dengan menempatkan Nginx *reverse proxy* sebagai satu-satunya container yang mengekspos port ke jaringan eksternal (host), komunikasi yang melewati batas perimeter jaringan Docker akan selalu melewati kanal TLS yang terenkripsi.

2.2.6 Framework MITRE ATT&CK

MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) merupakan *knowledge base* yang mengkatalogkan taktik, teknik, dan prosedur (TTP) yang digunakan oleh aktor ancaman. Dalam penelitian ini, teknik yang relevan adalah:

- Taktik: Credential Access (TA0006) dan Collection (TA0009)
- Teknik: Adversary-in-the-Middle (T1557), sub-teknik Network Sniffing (T1557.001)
- Mitigasi: Encrypt Sensitive Information (M1042), merekomendasikan penggunaan protokol enkripsi yang kuat untuk melindungi data dalam transit

2.2.7 OWASP Top 10:2025

Open Worldwide Application Security Project (OWASP) Top 10:2025 merupakan dokumen konsensus yang mengidentifikasi sepuluh risiko keamanan aplikasi web paling kritis. Kategori yang relevan dengan penelitian ini adalah:

- A02:2025-Cryptographic Failures: Kategori ini mencakup kegagalan dalam penerapan kriptografi yang mengakibatkan tereksposnya data sensitif. Contoh spesifik yang tercakup meliputi: transmisi data dalam *cleartext* (HTTP, SMTP, FTP), penggunaan algoritma kriptografi yang sudah usang atau lemah, dan tidak diterapkannya enkripsi pada data *in transit*. Mitigasi yang direkomendasikan oleh OWASP meliputi penggunaan TLS versi terbaru dengan *cipher suite* yang kuat untuk seluruh koneksi yang menangani data sensitif.

3. Metodologi Penelitian

3.1 Jenis dan Pendekatan Penelitian

Penelitian ini menggunakan pendekatan eksperimental kuantitatif dengan metode *before-after comparison*. Dua skenario pengujian dirancang untuk membandingkan kondisi transmisi data: (1) skenario *baseline* tanpa enkripsi (HTTP) dan (2) skenario *pasca-hardening* dengan TLS 1.3. Efektivitas mitigasi diukur berdasarkan kemampuan *packet analyzer* (Wireshark) dalam membaca konten payload pada masing-masing skenario.

3.2 Alat dan Bahan

3.2.1 Perangkat Keras

Komponen	Spesifikasi
Prosesor	Minimum Intel Core i5 / AMD Ryzen 5 (atau setara)
RAM	Minimum 8 GB
Penyimpanan	Minimum 20 GB ruang kosong

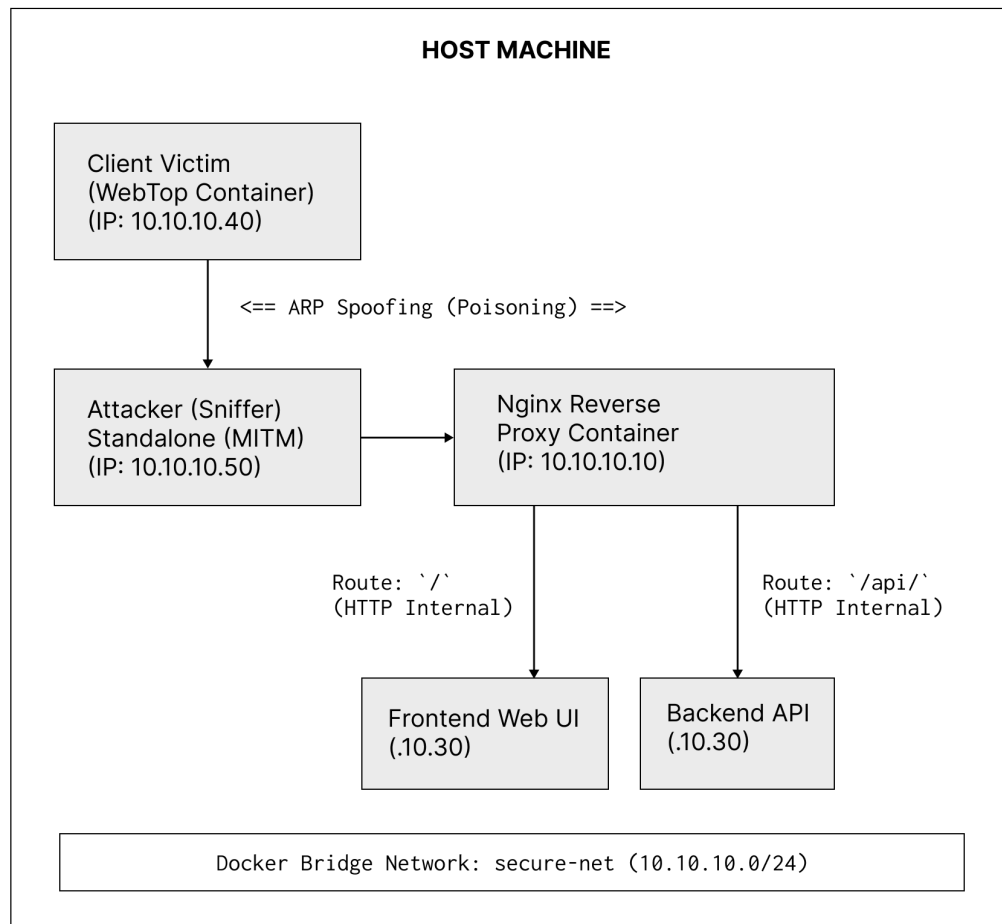
Jaringan	Loopback interface (localhost)
----------	--------------------------------

3.2.2 Perangkat Lunak

Perangkat Lunak	Versi	Fungsi
Docker Desktop	$\geq 4.x$	Platform container runtime
Docker Compose	$\geq 2.x$	Orkestrasi multi-container
Nginx	$\geq 1.25.x$	Reverse proxy dengan TLS termination
Python	≥ 3.10	Runtime backend API
Flask / FastAPI	Flask $\geq 3.x$ atau FastAPI ≥ 0.100	Framework backend REST API
OpenSSL	≥ 3.0	Pembuatan self-signed certificate
Wireshark	$\geq 4.x$	Packet analyzer untuk pengujian
cURL	$\geq 8.x$	HTTP client untuk pengiriman request

3.3 Arsitektur Sistem

Arsitektur yang dirancang terdiri dari tiga container Docker yang saling terhubung melalui *custom bridge network* bernama `secure-net`:



Designed in Figma

Gambar Diagram ARP Spoofing pada Arsitektur Microservices

Alur komunikasi dan Simulasi (Microservices & True MITM):

1. Seluruh container (termasuk *Client Victim* dan *Attacker*) beroperasi secara terisolasi di dalam jaringan internal *secure-net* yang menggunakan alokasi IP Statis Kelas A (10.10.10.0/24). Host machine tidak dapat mengakses API secara langsung melainkan melalui interface *WebTop*.
2. Serangan ARP Spoofing: Container *Attacker* (10.10.10.50) menjalankan *arp spoof* untuk meracuni *ARP Cache* dari *Client* dan *Proxy*. Hal ini mengecoh *switch* virtual Docker sehingga semua paket jaringan antara *Client* dan *Proxy* berbelok melewati *Attacker* terlebih dahulu (*Adversary-in-the-Middle*).

3. Pengguna mengakses kontainer *Client Victim* di port 3000. Dari dalam *browser* kontainer tersebut, Client mengirim HTTPS request. Paket ini ditangkap oleh *Attacker*, lalu diteruskan secara transparan ke Nginx reverse proxy (10.10.10.10).
4. Nginx melakukan terminasi TLS 1.3 dan memverifikasi bahwa koneksi menggunakan cipher suite TLS_AES_128_GCM_SHA256.
5. Nginx bertindak sebagai *API Gateway* yang melakukan *split routing*:
 - Request ke path / diteruskan ke container Frontend Web UI (10.10.10.30) untuk menampilkan halaman antarmuka pengguna.
 - Request pengiriman data ke path /api/ diteruskan ke container Backend API (10.10.10.20) untuk diproses.

3.4 Implementasi

3.4.1 Langkah 1: Pembuatan Self-Signed TLS Certificate

Sertifikat TLS di-*generate* menggunakan OpenSSL dengan perintah berikut:

```
openssl req -x509 -nodes -days 365 \  
-newkey rsa:2048 \  
-keyout ./certs/server.key \  
-out ./certs/server.crt \  
-subj \  
"/C=ID/ST=Jakarta/L=Jakarta/O=LabNetworkSecurity/CN=localhost"
```

Perintah ini menghasilkan pasangan *private key* (server.key) dan sertifikat *self-signed* (server.crt) dengan masa berlaku 365 hari, menggunakan RSA 2048-bit sebagai algoritma *key generation*.

3.4.2 Langkah 2: Konfigurasi Backend API

File app.py - Backend REST API menggunakan Flask:

```
from flask import Flask, request, jsonify  
  
app = Flask(__name__)
```



```

@app.route("/api/login", methods=["POST"])
def login():
    data = request.get_json()
    username = data.get("username")
    password = data.get("password")

    # Simulasi autentikasi sederhana
    if username == "admin" and password == "rahasia123":
        return jsonify({
            "status": "success",
            "message": "Autentikasi berhasil",
            "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.DEMO_TOKEN"
        }), 200
    else:
        return jsonify({
            "status": "failed",
            "message": "Kredensial tidak valid"
        }), 401

@app.route("/api/data", methods=["GET"])
def get_sensitive_data():
    return jsonify({
        "nim": "2024010001",
        "nama": "Test User",
        "nilai_uts": 85,
        "nilai_uas": 90
    }), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

File requirements.txt:

```

flask>=3.0.0
gunicorn>=21.2.0

```

File Dockerfile untuk backend:

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

```

```
COPY app.py .
EXPOSE 5000
CMD ["unicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

3.4.3 Langkah 3: Konfigurasi Nginx Reverse Proxy

File `nginx.conf`. Konfigurasi Nginx dengan TLS 1.3 restriktif:

```
server {
    listen 443 ssl;
    server_name localhost;

    # --- Sertifikat TLS ---
    ssl_certificate      /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;

    # --- HARDENING: Hanya TLS 1.3 ---
    ssl_protocols TLSv1.3;

    # --- HARDENING: Hanya cipher suite AES-128-GCM ---
    ssl_conf_command Ciphersuites TLS_AES_128_GCM_SHA256;
    ssl_prefer_server_ciphers on;

    # --- Security Headers ---
    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    # --- Reverse Proxy ke Backend API ---
    location / {
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

server {
    listen 80;
    server_name localhost;
    # Redirect seluruh HTTP ke HTTPS
```

```
    return 301 https://$host$request_uri;
}
```

Penjelasan konfigurasi kritis:

- `ssl_protocols TLSv1.3`, menolak seluruh koneksi yang menggunakan TLS 1.2 atau lebih rendah.
- `ssl_ciphers TLS_AES_128_GCM_SHA256`, membatasi *cipher suite* hanya pada AES-128-GCM, menolak AES-256-GCM dan ChaCha20-Poly1305.
- `return 301 https://...`, memastikan tidak ada koneksi HTTP yang diterima tanpa *redirect* ke HTTPS.

File Dockerfile untuk Nginx:

```
FROM nginx:1.25-alpine
RUN rm /etc/nginx/conf.d/default.conf
COPY nginx.conf /etc/nginx/conf.d/
COPY certs/ /etc/nginx/certs/
EXPOSE 443 80
```

3.4.4 Langkah 4: Orkestrasi dengan Docker Compose

File docker-compose.yml:

```
version: "3.9"

services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: backend-api
    networks:
      - secure-net
    expose:
      - "5000"
    # Port 5000 TIDAK di-publish ke host (hanya internal)

  nginx-proxy:
    build:
      context: ./nginx
```

```

    dockerfile: Dockerfile
    container_name: nginx-proxy
    ports:
      - "443:443"
      - "80:80"
    networks:
      - secure-net
    depends_on:
      - backend

networks:
  secure-net:
    driver: bridge

```

Aspek keamanan pada konfigurasi Docker Compose:

- Container `backend` hanya menggunakan `expose` (bukan `ports`), sehingga port 5000 tidak dapat diakses dari luar jaringan Docker.
- Hanya container `nginx-proxy` yang memetakan port ke *host machine*, memastikan seluruh akses eksternal melewati TLS termination point.
- Kedua container terhubung melalui *custom bridge network* `secure-net` yang terisolasi dari jaringan Docker default.

3.4.5 Langkah 5: Build dan Jalankan Environment

```

# Build seluruh container
docker-compose build

# Jalankan environment
docker-compose up -d

# Verifikasi container berjalan
docker-compose ps

```

3.4.6 Langkah 6: Verifikasi Koneksi TLS

```

# Verifikasi TLS 1.3 aktif dan cipher suite yang digunakan
openssl s_client -connect localhost:443 -tls1_3 2>/dev/null |
grep -E "Protocol|Cipher"

# Output yang diharapkan:
#   Protocol   : TLSv1.3

```

```
# Cipher      : TLS_AES_128_GCM_SHA256

# Verifikasi bahwa TLS 1.2 ditolak
openssl s_client -connect localhost:443 -tls1_2 2>/dev/null |
grep "alert"

# Output yang diharapkan:
# alert handshake failure
```

3.5 Prosedur Pengujian

Sebelum mengeksekusi skenario serangan, langkah pra-pengujian (*Sanity Check*) dilakukan untuk memastikan bahwa seluruh *container* telah berjalan dan mendapatkan IP Address yang sesuai dengan rancangan topologi. Verifikasi IP dilakukan dengan mengeksekusi perintah `hostname -i` (atau sejenisnya) dari terminal *host* ke dalam masing-masing *container*:

1. Nginx Reverse Proxy (`docker exec nginx-proxy hostname -i`):
10.10.10.10
2. Backend API (`docker exec backend-api hostname -i`): 10.10.10.20
3. Frontend Web UI (`docker exec frontend-web hostname -i`):
10.10.10.30
4. Client Victim (`docker exec client-victim hostname -i`): 10.10.10.40
5. Attacker MITM (`docker exec attacker hostname -i`): 10.10.10.50

Setelah topologi terverifikasi, pengujian dilanjutkan dalam dua skenario dengan memposisikan *container Attacker* (10.10.10.50) sebagai penyadap (MITM) yang mengeksekusi `tcpdump`, dan menyalurkan hasilnya secara langsung (*live streaming*) ke aplikasi Wireshark di mesin Host:

Skenario A: Baseline - Transmisi HTTP Tanpa Enkripsi

1. Konfigurasi Lingkungan: Nginx diatur untuk melayani HTTP *plaintext*, dan seluruh *container* dijalankan.
2. Memulai Penyadapan (Live Capture): Pada terminal Host, dieksekusi perintah untuk menyiarkan tangkapan jaringan dari dalam *container Attacker* langsung ke Wireshark:

```
docker exec -i attacker tcpdump -U -i eth0 host 10.10.10.40  
and not arp -w - | "C:\Program  
Files\Wireshark\Wireshark.exe" -k -i -
```

3. Simulasi Transmisi Data: Pengguna membuka *browser* dari mesin Host dan mengakses antarmuka WebTop Client di `http://localhost:3000`. Dari dalam *browser* WebTop (berperan sebagai *Client Victim*), pengguna mengakses halaman portal di `http://nginx-proxy` dan menekan tombol *Login* yang akan mengirimkan *request* POST (berisi kredensial) ke *Backend API*.
4. Analisis Paket: Pada jendela Wireshark yang sedang berjalan, diterapkan *display filter*: `http.request.method == POST`.
5. Ekstraksi Informasi: Fitur *Follow TCP Stream* digunakan pada paket tersebut untuk mengekstraksi konten *payload*.

Skenario B: Pasca-Hardening - Transmisi HTTPS dengan TLS 1.3

1. Konfigurasi Lingkungan: Nginx dikembalikan ke mode TLS 1.3 dengan AES-128-GCM, dan seluruh *container* di-*rebuild*.
2. Memulai Penyadapan (Live Capture): Perintah *piping tcpdump* ke Wireshark yang sama dengan Skenario A dieksekusi kembali pada *Command Prompt* Host.
3. Simulasi Transmisi Data (Aman): Melalui *browser* WebTop (`http://localhost:3000`), *Client Victim* mengakses portal secara aman di `https://nginx-proxy`. Pengguna kembali mengirimkan formulir kredensial melalui *Login*.
4. Analisis Paket: Pada jendela Wireshark, diterapkan *display filter*: `tls`.
5. Verifikasi Keamanan: Fitur *Follow TLS Stream* digunakan untuk memverifikasi bahwa *payload* aplikasi sepenuhnya tidak dapat diekstraksi dan hanya menampilkan byte acak (*Encrypted Application Data*).

3.6 Metrik Evaluasi

Metrik	Indikator Keberhasilan
Visibilitas payload (Skenario A)	Kredensial (username, password) terlihat dalam <i>cleartext</i> pada Wireshark
Visibilitas payload (Skenario B)	Kredensial tidak terlihat; hanya <i>encrypted application data</i> yang tampak
Versi protokol TLS	Wireshark menampilkan TLSv1.3 pada kolom <i>Protocol</i>
Cipher suite	Handshake menunjukkan TLS_AES_128_GCM_SHA256
Penolakan TLS lama	Koneksi TLS 1.2 dan lebih rendah menghasilkan <i>handshake failure</i>

4. Hasil dan Pembahasan

4.1 Hasil Implementasi Arsitektur

Implementasi arsitektur tiga container berhasil dilakukan menggunakan Docker Compose. Verifikasi status container menunjukkan seluruh layanan berjalan dengan normal:

[INSERT SCREENSHOT OUTPUT `docker-compose ps` YANG MENUNJUKKAN KEDUA CONTAINER BERSTATUS "Up" DI SINI]

Verifikasi koneksi TLS menggunakan `openssl s_client` mengonfirmasi bahwa Nginx reverse proxy telah dikonfigurasi dengan benar:

[INSERT SCREENSHOT OUTPUT `openssl s_client -connect localhost:443 -tls1_3` YANG MENUNJUKKAN "Protocol: TLSv1.3" DAN "Cipher: TLS_AES_128_GCM_SHA256" DI SINI]

Pengujian penolakan TLS 1.2 berhasil dilakukan:

[INSERT SCREENSHOT OUTPUT openssl s_client -connect localhost:443 -tls1_2 YANG MENUNJUKKAN "alert handshake failure" DI SINI]

4.2 Hasil Pengujian Skenario A: Baseline HTTP

Pada skenario tanpa enkripsi, Wireshark berhasil menangkap seluruh payload transmisi data dalam format *cleartext*. Melalui fitur *Follow TCP Stream*, kredensial autentikasi yang dikirimkan oleh client dapat dibaca secara utuh.

[INSERT SCREENSHOT WIRESHARK — CAPTURE PAKET HTTP POST KE /api/login DI SINI]

[INSERT SCREENSHOT WIRESHARK — FOLLOW TCP STREAM YANG MENAMPILKAN {"username":"admin","password":"rahasia123"} DALAM CLEARTEXT DI SINI]

Hasil ini mengonfirmasi temuan dari kedua jurnal referensi (Permana & Ramadhan, 2021; Darmawan et al., 2024) bahwa transmisi data melalui HTTP tanpa enkripsi bersifat rentan terhadap intersepsi. Pada *Follow TCP Stream*, terlihat:

- Request: Metode POST beserta header `Content-Type: application/json` dan body berisi `username` dan `password` dalam *cleartext*.
- Response: Body JSON berisi `token` autentikasi, `status`, dan `message` yang seluruhnya dapat dibaca tanpa hambatan.

Kondisi ini merepresentasikan skenario ancaman yang dipetakan oleh MITRE ATT&CK sebagai teknik T1557.001 (*Network Sniffing*) dan oleh OWASP sebagai A02:2025 (*Cryptographic Failures*).

4.3 Hasil Pengujian Skenario B: Pasca-Hardening TLS 1.3

Pada skenario *pasca-hardening*, Wireshark menangkap paket TLS yang terenkripsi. Meskipun *handshake* TLS dapat diamati (Client Hello, Server Hello), payload aplikasi tidak dapat dibaca.

[INSERT SCREENSHOT WIRESHARK — CAPTURE PAKET TLS 1.3 HANDSHAKE (CLIENT HELLO, SERVER HELLO) DI SINI]

[INSERT SCREENSHOT WIRESHARK — DETAIL TLS HANDSHAKE YANG MENUNJUKKAN CIPHER SUITE TLS_AES_128_GCM_SHA256 DI SINI]

[INSERT SCREENSHOT WIRESHARK — APPLICATION DATA YANG TERENKRIPSI (MENUNJUKKAN "Encrypted Application Data" BUKAN CLEARTEXT) DI SINI]

[INSERT SCREENSHOT WIRESHARK — FOLLOW TLS STREAM YANG MENAMPILKAN DATA TERENKRIPSI (BUKAN CLEARTEXT) DI SINI]

Hasil analisis terhadap *capture* paket menunjukkan:

1. TLS Handshake: Wireshark menampilkan fase *Client Hello* dengan daftar *cipher suite* yang didukung client, diikuti *Server Hello* yang memilih TLS_AES_128_GCM_SHA256 sebagai *cipher suite* aktif. Kolom *Protocol* pada Wireshark menampilkan TLSv1.3.
2. Application Data: Setelah *handshake* selesai, seluruh paket data selanjutnya ditampilkan sebagai *Application Data* dengan konten yang terenkripsi. Fitur *Follow TLS Stream* tidak dapat menampilkan *plaintext* karena Wireshark tidak memiliki akses ke *session key*.
3. Perbandingan langsung: Jika pada Skenario A credential "password":"rahasia123" terlihat secara eksplisit, pada Skenario B konten yang sama ditransmisikan sebagai deretan byte terenkripsi yang tidak dapat diinterpretasikan.

4.4 Analisis Komparatif Before-After

Parameter Pengujian	Skenario A (HTTP)	Skenario B (TLS 1.3)
Protokol yang terdeteksi Wireshark	HTTP	TLSv1.3

Visibilitas username	Terlihat (<i>cleartext</i>)	Tidak terlihat (terenkripsi)
Visibilitas password	Terlihat (<i>cleartext</i>)	Tidak terlihat (terenkripsi)
Visibilitas response token	Terlihat (<i>cleartext</i>)	Tidak terlihat (terenkripsi)
Follow Stream	TCP Stream: seluruh payload terbaca	TLS Stream: <i>encrypted data</i>
Cipher suite	Tidak ada	TLS_AES_128_GCM_SHA256
Forward Secrecy	Tidak ada	ECDHE (wajib di TLS 1.3)

Tabel di atas menunjukkan bahwa seluruh data sensitif yang sebelumnya terekspos pada Skenario A telah berhasil diproteksi pada Skenario B. Tidak ada satu pun informasi *plaintext* dari payload aplikasi yang dapat diekstraksi melalui Wireshark setelah *hardening* TLS 1.3 diterapkan.

4.5 Pemetaan Terhadap MITRE ATT&CK

Pemetaan hasil pengujian terhadap framework MITRE ATT&CK disajikan dalam tabel berikut:

Komponen MITRE ATT&CK	Detail	Status Mitigasi
Taktik	Credential Access (TA0006), Collection (TA0009)	-
Teknik	Adversary-in-the-Middle (T1557)	-
Sub-teknik	Network Sniffing (T1557.001)	Berhasil dimitigasi

Prosedur Simulasi	Sniffing kredensial REST API via Wireshark	Skenario A: berhasil; Skenario B: gagal (terenkripsi)
Mitigasi yang Diterapkan	Encrypt Sensitive Information (M1042)	Diimplementasikan via TLS 1.3 + AES-128-GCM
Validasi	Data yang ditransmisikan tidak dapat dibaca oleh <i>packet analyzer</i> pasca-mitigasi	Terkonfirmasi

Berdasarkan pemetaan di atas, implementasi TLS 1.3 dengan cipher suite AES-128-GCM secara langsung memenuhi rekomendasi mitigasi M1042 dari MITRE ATT&CK. Teknik serangan T1557.001 (*Network Sniffing*) yang pada Skenario A berhasil mengekstraksi kredensial, pada Skenario B hanya menghasilkan data terenkripsi yang tidak dapat diinterpretasikan tanpa *session key*.

4.6 Pemetaan Terhadap OWASP Top 10:2025

Komponen OWASP	Detail	Status
Kategori	A02:2025 - Cryptographic Failures	-
Kerentanan yang Diuji	Transmisi data sensitif tanpa enkripsi (cleartext over HTTP)	Teridentifikasi pada Skenario A
Solusi yang Diterapkan	Enkripsi transport layer menggunakan TLS 1.3 dengan cipher suite AES-128-GCM	Diimplementasikan pada Skenario B
Kepatuhan Rekomendasi OWASP		
1. Gunakan TLS terbaru	TLS 1.3 (RFC 8446)	Terpenuhi

2. Gunakan cipher suite kuat	TLS_AES_128_GCM_SHA256 (AEAD)	Terpenuhi
3. Terapkan HSTS	Strict-Transport-Security header aktif	Terpenuhi
4. Nonaktifkan koneksi HTTP	Redirect 301 HTTP → HTTPS	Terpenuhi

Implementasi yang dilakukan pada penelitian ini memenuhi seluruh rekomendasi OWASP untuk mitigasi risiko A02:2025. Penggunaan TLS 1.3 secara eksklusif (tanpa *fallback* ke versi lebih rendah) dan pembatasan cipher suite ke AEAD-only menutup celah keamanan yang menjadi dasar klasifikasi *Cryptographic Failures* oleh OWASP.

4.7 Perbandingan dengan Penelitian Sebelumnya

Hasil pengujian dalam penelitian ini secara empiris menjawab gap yang ditinggalkan oleh kedua jurnal referensi:

Aspek	Permana & Ramadhan (2021)	Darmawan et al. (2024)	Penelitian Ini
Temuan utama	HTTP rentan terhadap sniffing	Jaringan rentan disadap	TLS 1.3 + AES-128-GCM efektif mencegah sniffing
Tindak lanjut	Rekomendasi enkripsi (tanpa implementasi)	Tidak ada	Implementasi + pengujian + validasi
Bukti mitigasi	Tidak ada	Tidak ada	Wireshark capture before-after

Pemetaan standar	Tidak ada	Tidak ada	MITRE T1557.001 + OWASP A02:2025
Reproduktifitas	Terbatas (tidak ada kode)	Terbatas	Penuh (Docker Compose + konfigurasi lengkap)

Penelitian ini membuktikan bahwa rekomendasi generik terkait enkripsi yang diajukan oleh penelitian sebelumnya memerlukan spesifikasi lebih lanjut agar efektif. Tidak cukup hanya mengaktifkan HTTPS; konfigurasi harus secara eksplisit membatasi versi TLS ke 1.3 dan memilih cipher suite AEAD untuk menutup seluruh vektor serangan pada layer transport.

5. Kesimpulan dan Future Work

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, penelitian ini menghasilkan tiga kesimpulan utama yang menjawab rumusan masalah:

1. Arsitektur keamanan berbasis TLS 1.3 berhasil diimplementasikan dalam ekosistem Docker *Microservices* yang memisahkan komponen Frontend, Backend API (Flask), Nginx *reverse proxy* sebagai TLS termination point, serta simulasi Client Victim (*WebTop Browser*). Konfigurasi Nginx secara restriktif membatasi protokol pada TLSv1.3 saja dengan cipher suite tunggal TLS_AES_128_GCM_SHA256, menolak seluruh koneksi yang menggunakan versi TLS lebih rendah.
2. Efektivitas konfigurasi divalidasi secara langsung melalui skenario penyerangan *True MITM* (ARP Spoofing) menggunakan container *Attacker* mandiri berbasis Kali Linux. Pada skenario baseline (HTTP), seluruh kredensial autentikasi yang disadap dapat diintersepsi dalam format *cleartext* melalui Wireshark. Pada skenario pasca-hardening (TLS 1.3), Wireshark hanya menangkap *Encrypted Application Data* tanpa kemampuan untuk membaca payload. Hal ini menunjukkan bahwa konfigurasi TLS 1.3 dengan

AES-128-GCM efektif mencegah serangan *network sniffing* meskipun lalu lintas jaringan telah sepenuhnya berbelok (terkompromi) di layer 2.

3. Pemetaan terhadap framework standar industri mengonfirmasi bahwa implementasi yang dilakukan memenuhi rekomendasi mitigasi M1042 (*Encrypt Sensitive Information*) dari MITRE ATT&CK untuk mengatasi teknik T1557.001, serta memenuhi seluruh panduan mitigasi OWASP untuk risiko A02:2025 (*Cryptographic Failures*) termasuk penggunaan TLS terbaru, cipher suite kuat (AEAD), header HSTS, dan penolakan koneksi HTTP.

5.2 Future Work

Penelitian ini memiliki beberapa potensi pengembangan untuk riset selanjutnya:

1. Implementasi Mutual TLS (mTLS): Menambahkan autentikasi dua arah di mana backend API juga memverifikasi sertifikat client, sehingga tidak hanya koneksi yang terenkripsi tetapi juga identitas client yang terjamin. Pendekatan ini relevan untuk arsitektur *zero-trust* di mana setiap komunikasi *service-to-service* memerlukan verifikasi identitas.
2. Pengujian pada jaringan multi-node: Penelitian ini terbatas pada lingkungan *localhost*. Pengujian lanjutan dapat dilakukan pada jaringan multi-node menggunakan Docker Swarm atau Kubernetes untuk mengevaluasi efektivitas TLS 1.3 pada skenario *inter-node communication* yang lebih realistis.
3. Analisis performa (*performance benchmarking*): Mengukur *overhead* latensi dan *throughput* yang ditimbulkan oleh enkripsi TLS 1.3 dibandingkan dengan HTTP menggunakan *benchmarking tools* seperti Apache Benchmark (ab) atau wrk. Analisis ini penting untuk mengevaluasi *trade-off* antara keamanan dan kinerja pada beban produksi.
4. Pengujian vektor serangan tambahan: Memperluas cakupan pengujian ke teknik MITRE ATT&CK lain yang relevan, seperti T1040 (*Network Sniffing* tanpa AitM), T1557.002 (*ARP Cache Poisoning*), atau pengujian terhadap serangan *SSL stripping* untuk memvalidasi efektivitas header HSTS.
5. Integrasi Certificate Authority (CA) dan OCSP Stapling: Mengganti sertifikat *self-signed* dengan sertifikat dari CA publik (misalnya Let's Encrypt) dan mengaktifkan OCSP

stapling pada Nginx untuk menyediakan mekanisme validasi revokasi sertifikat secara *real-time*.

6. Automasi hardening menggunakan Infrastructure as Code (IaC): Mengembangkan template Ansible atau Terraform yang mengotomasi seluruh proses konfigurasi TLS hardening, sehingga dapat diterapkan secara konsisten pada lingkungan produksi berskala besar.

Daftar Pustaka

- [1] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force (IETF), Agustus 2018. [Daring]. Tersedia: <https://datatracker.ietf.org/doc/html/rfc8446>
- [2] MITRE Corporation, "Adversary-in-the-Middle: Network Sniffing, Sub-technique T1557.001 – MITRE ATT&CK®," 2024. [Daring]. Tersedia: <https://attack.mitre.org/techniques/T1557/001/>
- [3] MITRE Corporation, "Encrypt Sensitive Information, Mitigation M1042 – MITRE ATT&CK®," 2024. [Daring]. Tersedia: <https://attack.mitre.org/mitigations/M1042/>
- [4] OWASP Foundation, "OWASP Top 10:2025 – A02:2025 Cryptographic Failures," 2025. [Daring]. Tersedia: https://owasp.org/Top10/A02_2025-Cryptographic_Failures/
- [5] A. T. Permana dan A. F. Ramadhan, "Analisis Keamanan Jaringan pada Layanan WiFi dengan Menggunakan Wireshark," *Infoman's: Jurnal Ilmu-ilmu Informatika dan Manajemen*, 2021.
- [6] M. A. Darmawan, R. Ningsih, dan A. J. Wahidin, "Analisis Keamanan Jaringan terhadap Sniffing Menggunakan Wireshark," *Just IT: Jurnal Sistem Informasi, Teknologi Informasi dan Komputer*, vol. 14, no. 3, 2024.
- [7] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. London: Pearson Education, 2020.

- [8] National Institute of Standards and Technology (NIST), "SP 800-52 Rev. 2: Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations," Agustus 2019. [Daring]. Tersedia: <https://doi.org/10.6028/NIST.SP.800-52r2>
- [9] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Disertasi Doktorat, University of California, Irvine, 2000. [Daring]. Tersedia: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [10] Docker Inc., "Docker Networking Overview – Docker Documentation," 2024. [Daring]. Tersedia: <https://docs.docker.com/network/>
- [11] Nginx Inc., "NGINX Reverse Proxy – NGINX Documentation," 2024. [Daring]. Tersedia: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/>
- [12] Nginx Inc., "Configuring HTTPS Servers – NGINX Documentation," 2024. [Daring]. Tersedia: https://nginx.org/en/docs/http/configuring_https_servers.html
- [13] M. Dworkin, "SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," National Institute of Standards and Technology (NIST), November 2007. [Daring]. Tersedia: <https://doi.org/10.6028/NIST.SP.800-38D>
- [14] G. Combs et al., "Wireshark User's Guide – Version 4.x," Wireshark Foundation, 2024. [Daring]. Tersedia: https://www.wireshark.org/docs/wsug_html_chunked/
- [15] D. Cooper et al., "SP 800-52 Rev. 2: Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations," National Institute of Standards and Technology (NIST), Agustus 2019. [Daring]. Tersedia: <https://csrc.nist.gov/publications/detail/sp/800-52/rev-2/final>
- [16] T. Dierks dan E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," RFC 5246, Internet Engineering Task Force (IETF), Agustus 2008. [Daring]. Tersedia: <https://datatracker.ietf.org/doc/html/rfc5246>

[17] B. Beurdouche et al., "A Messy State of the Union: Taming the Composite State Machines of TLS," dalam *Proceedings of the 2015 IEEE Symposium on Security and Privacy*, San Jose, CA, 2015, hlm. 535–552. [Daring]. Tersedia: <https://doi.org/10.1109/SP.2015.39>

[18] HTTPS.in. (2019, November 14). *TLS 1.3 - Fast and more Secure | Here's everything you need to know*. HTTPS.in Blog. Diperoleh 17 Juni 2026, dari <https://www.https.in/blog/tls-1-3/>